

Structures & Functions

There are 3 ways to pass structure as an argument to a function

↳ by passing an individual structure member.

↳ by passing entire structure

↳ by passing pointer to the structure.

Passing individual structure member

```
void main()
{
    struct s
    {
        int rollno;
        char nm[20];
    };
    struct s s1;
    void stud(int r, char n[20])
    {
        printf("In Rollno=%d \t name=%s", r, n);
    }
    // obj
    // op: enter data a
    // structure Rollno=2 name=a
    printf("enter data");
    scanf("%d %s", &s1.rollno, s1.nm);
    stud(s1.rollno, s1.nm);
}
```

- becomes unmanageable & inefficient when the structure size is large.

Passing entire structure

Here a copy of entire structure is passed. Since the function is working on a copy of the structure any changes made to structure members within the function are not reflected in the original structure.

e.g struct s
{
int rollno;
char nm[20];
};

```
void main()
{
    void stud(struct s);
    printf("enter data");
    scanf("%d %s", &s1.rollno, s1.nm);
    stud(s1);
}
```

```
void stud(struct s s2)
{
    printf("Rollno=%d \t name=%s",
           s2.rollno, s2.nm);
}
```

- In this case it is necessary to define the structure type struct s outside main(). so that it becomes known to all functions in the program.

Passing Pointers to structure

A structure can be passed to a function as an argument through pointers i.e address of the structure is passed.

```
e.g struct s
{
    int rollno;
    char nm[20];
};
void main()
{
    struct s s1;
    void stud(struct s *s2)
    {
        scanf("%d", &s2->rollno);
        scanf("%s", s2->nm);
    }
    stud(&s1);
    printf("Rollno=%d \t name=%s", s1.rollno, s1.nm);
}
```

Arrow operator (->): made up of a minus sign and greater than sign

e.g struct s
{
char nm[30];
int rno;
} P[2], *PI;

PI is pointer to data objects of the struct s
PF P; // assign address of oth element of P to PI i.e PI points to P[0].

Accessing members of structure

$P \rightarrow nm$ $P \rightarrow rno$

$P++$ will point to next record i.e. $P[1]$

We can also use

$(*P).nm$ parentheses is necessary as '.' has higher precedence.

Pointer to structure Variable

print record of 2 students

```
struct stud
{
    char nm[20];
    int rno;
};

void main()
{
    struct stud s[2], *p;
    int i;
    p = s;

    while (i < 2)
    {
        scanf("%s %d", p->nm, &p->rno);
        p++;
    }
    p = s;
    while (i < 2)
    {
        printf("In name = %s \t rollno = %d",
               (*p).nm, (*p).rno);
    }
}
```

Pointer as a member of structure

void main()

```
{
    struct s
    {
        char *nm;
        int rno;
    } s1, *p;
```

char nm1[] = "Soma";

int rno1 = 23;

$P \rightarrow nm = nm1$;

$P \rightarrow rno = \&rno1$;

printf("Name: %s \t", $P \rightarrow nm$);

printf("Rollno: %d \n", $*P \rightarrow rno$);

}

```
s1.nm = nm1;
s1.rno = &rno1;
printf("Name: %s", s1.nm);
printf("In Rollno: %d", *s1.rno);
```

Copying & Comparing structure

Two variables of the same structure type can be compared the same way as ordinary variables.

e.g. struct s s1, s2;

$s2 = s1$ assign s2 to s1 (since structure elements are stored in contiguous memory)

$s1 == s2$ Compare all members of s1 & s2 and return 1 if they are equal, 0 otherwise.

$s1 != s2$ return 1 if all the members are not equal, 0 otherwise.

e.g. struct stud

```
{
    char nm[30];
    int rno;
};
```

void main()

{

struct stud s1 = {"A", 4};

struct stud s2;

$s2 = s1$;

printf("In name = %s \t rollno = %d", s2.nm, s2.rno);

}

Self-referential structures

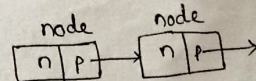
- self-referential structures contain a pointer member that points to a structure of the same structure type.

e.g. struct node

```
{
    int n;
```

struct node *p;

}



- p is a pointer member of the structure which is a pointer to the same structure in which it is declared or stores the address of the structure of same type.

Self-referential structures can be linked together to form useful data structures such as linked lists, queues, stacks & trees.

```
struct node
{
    int n;
```

```
struct node *p;
    } n1, n2, n3;
```

```
void main()
```

```
{
    n1.p = &n2;
```

```
    n2.p = &n3;
```

```
    n3.p = NULL;
```

```
    scanf("%d %d %d", &n1.n, &n2.n, &n3.n);
```

```
    printf("In n1.n = %d It n1.p = %u", n1.n, n1.p);
```

```
    printf("In n2.n = %d It n2.p = %u", n2.n, n2.p);
```

```
    printf("In n3.n = %d It n3.p = %u", n3.n, n3.p);
```

```
}
```

typedef: used for defining new datatype.

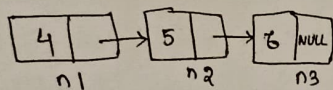
```
e.g typedef int mck;
    mck sub1, age;
```

```
* typedef struct
{
    int kno;
```

```
    } s1;
```

- When typedef is used to name a structure the structure tag name is not necessary.

- Here s1 is used for defining the structure & s2 has same structure as the s1.



o/p 3 4 5

n1.n = 3 n1.p = 65520

n2.n = 4 n2.p = 65512

n3.n = 5 n3.p = 0

UNION: union is same as structure.

- In structure each member has its own memory location. Whereas members of unions has same memory location. We can assign value to only one member at a time.

- When union is declared, Compiler automatically allocates a memory locations to hold the largest data type of members in the union. Union is used for saving of memory and it is useful when it is not necessary to assign the values to all the members of the union at a time.

Syntax: union unionname
{
 members
 } variable name;

e.g union item
{
 int m;
 float x;
 } i1;

union variable can also be declared as
union unionname variablename
e.g union item i2;

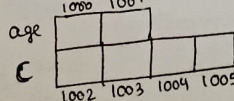
Difference between structure & union

struct emp

```
{
    int age;
```

```
    char c[4];
```

```
    } s1;
```

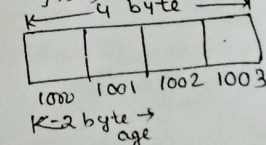


union emp

```
{
    int age;
```

```
    char c[4];
```

```
    } s1;
```



To access union member

- To access directly union member we use dot(.) operator
- To access through pointer we use (→) operator.

program to access union member

void main()

```
{
union res
{
    int m;
    char grade;
} r;
```

printf("Size of union: %d\n", sizeof(r));

r.m = 90;

printf("mark = %d |t grade = %c\n", r.m, r.grade);

r.grade = 'A';

printf("mark = %d |t grade = %c\n", r.m, r.grade);

}

o/p - size of union: 2

mark = 90 grade = ^ (garbage value)

mark = 1935 grade = A
(garbage value)

using pointer

- union res

```
{
    int m;
    char grade;
} *r;
```

r → m = 75;

printf("mark = %d |t grade = %c", r → m, r → grade);

r → grade = 'A';

P.f ("mark = %d |t grade = %c", r → m, r → grade);

o/p. mark = 75 grade = - (garbage value)

mark = (garbage value) grade = A

- union may be used in all places where structure is allowed. A structure may be a member of union or union may be a member of structure.

e.g. union res

```
{
    int m;
    char grade;
} s;
struct res1
{

```

char nm[15];

union res r;

sizeof(d) = 17

struct res1

char nm[15];

sizeof(d) = 17

union res

{

int m;

char grade;

} r;

} d;

union & structure

i) union & structure are almost similar.

union stud

```
{
    int roll;
```

char nm[10];

} s1;

struct stud

```
{
    int roll;
```

char nm[10];

} s1;

2) multiple members are collected together under same name.

3) All union members occupy same memory area.

4) only one member will be active at a time.

COMMAND LINE ARGUMENT

- A command line argument is the information that follows the name of the program on the command line of the OS.

- Command-line arguments are used to pass information to a program when you run it.

- Two built-in parameters are used to main() receive command line arguments. They are optional.

Syntax: main(int argc, char *argv[])

argument counter → argc is an integer. It holds the no. of arguments on command line. Since the name of the program always corresponds to the first argument, it is always at least 1.